

LumoTree: Path-Parallel Speculative Verification for Hybrid Language Models

Zhiyuan Ma

zhiyuanma0520@gmail.com

Abstract—Tree speculative decoding for long-running coding agents must preserve coherent continuation across repeated generation and tool calls. We present LumoTree, a tree-verification system for Gated DeltaNet hybrid models. LumoTree executes recurrent paths in parallel, reuses state tiles across updates within each path, and coordinates the accepted continuation across recurrent, convolution, and attention state. A shared logical tree connects temporary branch computation to native recurrent replay, convolution-history gathering, and attention-cache remapping. Fused candidate selection, GPU-resident acceptance, and grouped split-K attention reduce repeated work and host orchestration. On Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both shared SWE-bench Verified Astropy tasks, the best recorded tree run achieves 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. Component tests verify candidate-selection parity and measure attention repeatability and numerical error.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

LumoTree executes recurrent paths in parallel, reuses state tiles across updates within each path, and coordinates the accepted continuation across recurrent, convolution, and attention state. An ancestry-aware attention path and a device acceptance walk use the same logical tree as recurrent verification and state publication. A shared descriptor carries ancestry and logical-to-physical mappings through each stage [6], [7]. This system boundary matters because a verifier can compute plausible candidate logits yet publish the wrong state for the next iteration.

The central execution choice is to verify branches in temporary state and replay only the selected path into the native running state. The verifier scans branch paths, while the fixed-shape committer replays accepted operands through native GDN updates in a captured graph. Their finite-precision agreement is a separate requirement, not assumed from the recurrence. This choice exposes concrete optimization questions: how much branch state to cache, when to recompute ancestry, how

to organize independent GPU work, and how to keep full-vocabulary probability arithmetic on the device. The attention layout and accepted-state mappings constrain those changes as much as the recurrent equation does.

The paper studies three coupled design questions in recurrent-hybrid tree verification. *First*, how can branch-local computation produce one coherent continuation across recurrent, convolution, and attention state? We define a shared logical tree and coordinate state publication through its logical-to-physical mapping. *Second*, how should verification trade temporary state storage against repeated computation? We use path-parallel scans with shared boundary states and replay the accepted path into persistent state, building on established chain-scheduling and replay mechanisms [8], [9]. *Third*, how can this organization reduce serving overhead? We combine fused candidate selection, device-resident acceptance, and grouped split-K attention with a spine-contiguous KV layout. Component validation and coding-agent workloads characterize the resulting system; matched ablations remain necessary to attribute gains to individual choices.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, and a task can require many model requests. Performance therefore depends on useful completed work as well as token production. Our workload evaluation uses explicitly identified Astropy tasks from SWE-bench Verified [10]. We report the model, serving route, task cohort, and measurement definition together; a decode-rate change is not a task-completion speedup. Candidate-selection and attention tests complement the agent-workload measurements.

Tree verification and compact accepted-state reconstruction have direct prior art. Section II positions our implementation against those mechanisms. The architecture and kernel design are developed in Sections III–IV, and Section V separates optimization mechanisms from their measured scope. Measurement provenance is documented in Appendix A.

II. BACKGROUND AND RELATED WORK

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [11]–[13]. EAGLE-3 and DFlash further change the drafter, through feature-conditioned autoregression and

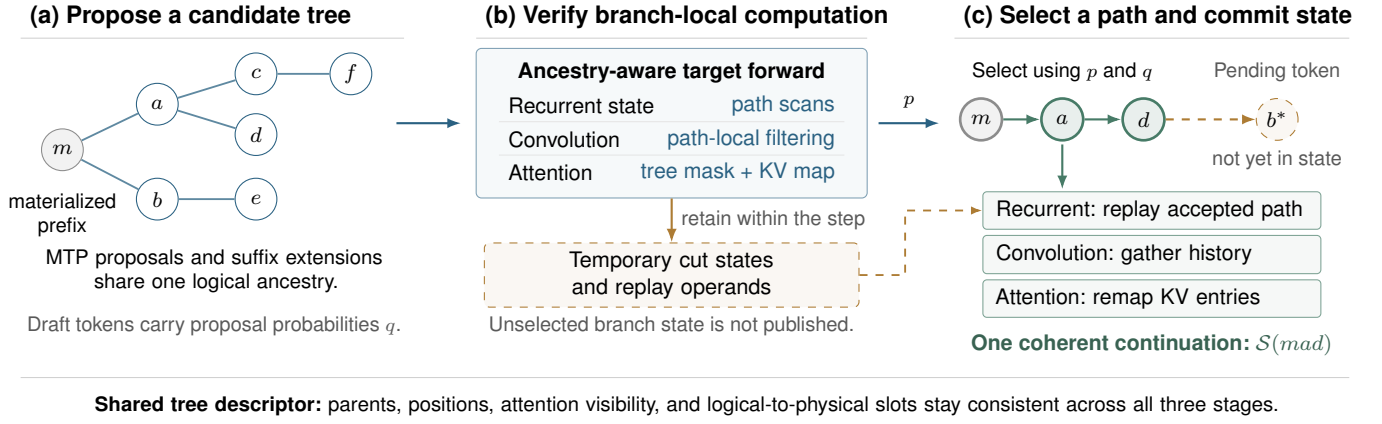


Fig. 1. Tree verification and accepted-path commitment. (a) Drafting provides a candidate tree and proposal probabilities q . (b) Branch-local computation produces target probabilities p and retains temporary state. (c) Sampling selects a materialized path (here a, d); recurrent replay, convolution gathering, and KV remapping publish $\mathcal{S}(mad)$. The newly sampled token b^* remains pending. One descriptor keeps logical ancestry and physical storage aligned.

block-parallel diffusion proposals respectively [14], [15]. These drafting mechanisms are distinct from the recurrent verifier and state-publication choices studied here.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [16], [17]. Marconi addresses prefix-cache admission and eviction for hybrid models [18]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [19], [20]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. Flash-Decoding splits the KV sequence and combines partial attention results; FlashInfer develops customizable attention with head/query fusion and split-KV execution; DeFT specializes KV grouping and partitioning to trees [21]–[23]. FastTree groups queries with shared contexts to reuse KV tiles and partitions tree work to control execution cost [24]. These attention methods establish precedents for on-chip reuse and tree-aware partitioning. Our modified FlashAttention-2 kernel combines ancestry-aware decode, grouped split-K execution, and a physical layout coordinated with accepted-path publication [6], [25].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [26]. Snakes and Ladders introduces activation replay for SSM speculative decoding, and STree also uses replay to recover continuation state [5], [27]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [28], [29]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Trees from Marginals (Weaver) combines a factorized drafter with an autoregressive adapter. Its GDN verifier uses an ancestor-masked triangular solve and replays the selected path through a short recurrence at commit time; padded verification shapes support CUDA graphs [9]. Thus read-only speculative verification followed by accepted-path replay is direct prior art, even when the verification arithmetic differs from ours.

SpecLA keeps value tiles of recurrent state on chip across updates within a chain and schedules independent ready chains in parallel. Boundary states connect dependent chains. This directly overlaps both the state reuse and path scheduling in LumoTree. SpecLA buffers accepted factors for a later fused update, whereas our publication stage replays recorded operands through native GDN updates into running rows. Its evaluation uses a pure GDN model; our serving boundary additionally coordinates convolution and softmax attention [8]. SpecLA’s token-replay baseline regenerates factors, so its cost cannot be assigned to our replay of cached operands.

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It reuses intermediate tiles on chip, stores compact update factors, and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-72B on GB10 and replay of recorded OpenHands agent sessions [30]. This is a direct hybrid-serving comparison; its workload replay differs from running the coding agent on tasks, as in our evaluation.

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the source correction. Tree depth d gives $G^{d+1} = 0$, so $U = \sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [30]. Our path preserves sequential updates and replays accepted operands; Section V-B explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular

TABLE I
RECURRENT VERIFICATION AND CONTINUATION POLICIES. THESE
MECHANISM COMPARISONS DO NOT RANK THE SYSTEMS BY
PERFORMANCE.

Method	Verification work	Continuation state
Weaver [9]	Ancestor-masked triangular solve	Short selected-path recurrence
SpecLA [8]	Resident state tiles; parallel chains	Buffered factors; later fused update
Bole [30]	Finite-Neumann correction solve	Compact factors; state reconstruction
TreeWY [32]	Triangular solve in fused kernel	Prior-path reconstruction with next verify
ReplaySSM [33]	Corrected inputs; cached transitions	Deferred state update
OneLA [34]	Shared-base projections across beams	Compact per-beam transition records
LumoTree	Resident state tiles; parallel paths	Native replay; coordinated convolution and KV publication

system and reconstructs the accepted state from pseudo-values. The author implementation fuses reconstruction of the previous accepted state with the next verification, reusing a resident state tile. Its GDN tree kernel supports CUDA graph capture; the paper’s wider-tree fallback to piecewise execution concerns ancestry masks in the softmax-attention integration [31], [32]. Its B200 evaluation uses greedy verification and disabled prefix caching. A comparison must therefore include fused commit work and distinguish kernel support from whole-model execution policy.

ReplaySSM caches recurrent inputs and delays state materialization. Its GDN path uses corrected inputs from a triangular solve, with commitment represented through buffer progress and later state updates [33]. **OneLA** addresses a related state-sharing problem in recommendation beam search: compact transition records and ancestry indices allow projection computation without reconstructing a full state for each beam, and a fused kernel shares the prompt-derived state across beams [34]. Its beam-search output and commitment policy differ from speculative acceptance. Table I compares state lifetimes; published speed ratios from different models and workloads are not combined into a ranking.

D. Design focus

LumoTree connects path-parallel recurrent computation to one accepted continuation across the hybrid model. The shared descriptor links path boundaries, convolution ancestry, attention visibility, physical KV slots, and the next drafter input. Native replay preserves the serving system’s recurrent-state ownership while selected-path gathering and remapping publish the other state components. Spine-first KV placement controls the physical attention reduction layout within this design, building on established floating-point reduction and batch-invariance concerns [35]. Evaluation separates component behavior from complete agent-workload performance.

III. TREE-VERIFIER ARCHITECTURE

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$\mathcal{S}(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(\mathcal{S}(m), b)$, not a claim that $\mathcal{S}(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 2 illustrates the three state surfaces coordinated by the verifier [6], [7].

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

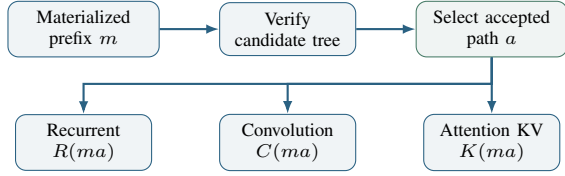
$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. A descriptor shared by verification and commitment

Figure 1 shows how a shared descriptor connects drafting to continuation. Native multi-token prediction (MTP) supplies a principal chain and runner-up candidates; suffix prediction extends selected paths within a bounded candidate capacity [36], [37]. Combining retrieval and neural drafting has prior art, including SAM Decoding [38]. The verifier derives attention visibility, physical packing, path scheduling, and the acceptance walk from the same logical topology. Padding permits reusable execution shapes while remaining invisible to attention and selection. The particular tree geometry used in our experiments is specified in Section VI.



Same selected materialized prefix on all three surfaces; correction token b may remain pending for the next forward.

Fig. 2. The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled correction or bonus token can remain pending until the next forward.

The attention layout places the principal spine first in the physical suffix and permutes KV write locations and ancestry-mask key columns while retaining logical query-row and position order. Selection remains in logical-node space; accepted-path publication maps those nodes back to their physical slots. This distinction prevents a layout optimization from silently changing which key/value entries a branch reads. Recurrent paths begin at the native pre-tree state or a transient cut state; the device committer publishes the selected recurrent and convolution state into native running rows and re-linearizes its attention KV entries [6], [7].

Algorithm 1 separates candidate verification, path selection, and state publication. Every publication operation uses the same materialized path A . A newly sampled token b^* can be emitted while remaining pending: its state is created by the next target forward.

IV. GPU SCAN AND ACCEPTED-PATH REPLAY

A. Path-parallel verification and state lifetime

The GDN verifier decomposes the tree into dependent path groups, a scheduling family also studied by SpecLA [8]. A first path starts from the request’s pre-tree state and exports the cut states needed by downstream paths. The second level evaluates independent branch paths from those cut states. Each GPU program covers one path, head, and value tile, retaining the full key dimension in its working state tile. Sequential updates reuse this tile through the path loop rather than loading a full recurrent state for every node. Candidate outputs continue through the model; rounded operands are recorded for later accepted-path replay [6]. The source expresses this local reuse; register allocation and spills remain compiler- and geometry-dependent.

Cut states and operand rings are temporary within a verification step. The durable continuation resides in the request-owned native recurrent and convolution running rows. Replay starts from the pre-tree running state and publishes the accepted frontier back to those rows; prefix-cache snapshots and restores use that native layout. Static work buffers can be reused by CUDA graphs, but they do not replace the request’s persistent state with a branch-leaf pointer.

The two levels execute in separate GPU launches and exchange only the boundary states required by downstream

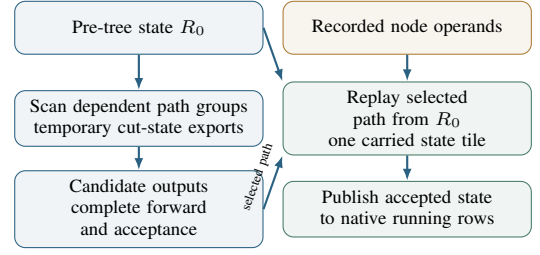


Fig. 3. State lifetime in the path-scan verifier. Temporary cut states connect path groups; recorded operands support selected-path replay. Only the accepted frontier becomes native persistent state.

paths. This schedule exposes parallel branch work while retaining the state-transfer cost at the decomposition boundary.

B. Sequential arithmetic and accepted-path replay

Verification uses sequential path scans with fp32 carried state, raw gate inputs, and in-kernel query/key normalization. Accepted-path replay uses native fused GDN updates with recorded accepted operands and in-place publication to the fp32 state bank. Its graph contains one native update call per GDN layer. Query/key normalization, beta conversion, gate evaluation, and output casts can differ between these implementations; agreement is not guaranteed by their common real-arithmetic recurrence [6].

The sampling stage produces accepted node indices and path lengths on the GPU. Recurrent replay, convolution publication, and attention-KV publication consume this same path. Graph capture reduces host launch overhead for the sequence of per-layer native updates; each layer retains its own update kernel.

C. Temporary state versus recomputation

Path decomposition caches the states needed where downstream paths branch, then reuses them across independent work. If a program exports N_c cut states with value-tile width B_v and key dimension d_k , their fp32 payload is

$$M_{\text{cut}} = 4N_c B_v d_k \text{ bytes.} \quad (4)$$

This counts one head and value tile’s logical export payload. The current reusable scratch allocation reserves a full matrix slot for each actual node even though only required cut states are written; allocated bytes therefore differ from state traffic. Recomputing ancestry could remove some exports at the cost of repeating updates. Exporting every candidate state would instead trade more writes for less recomputation. The two-level path schedule occupies a particular point in this design space.

Accepted-path replay carries one state tile through a bounded path loop. Its carried-state footprint is $O(B_v d_k)$ in tree size, while the recorded operands and candidate outputs scale with the verification shape. Static padding makes graph buffers and kernel geometry reusable, but padded rows must remain invisible to attention and selection. Neither a shorter logical path nor fewer state writes alone establishes a faster complete request.

The closest scheduling comparison is SpecLA: both systems carry state tiles through sequential paths and parallelize ready

Algorithm 1 Tree verification and native continuation

Input: Native continuation state; candidate tree T ; proposal probabilities q

Output: Emitted tokens; updated native state; next drafter input; any pending token

Verify candidates

- 1: Map T to physical slots, positions, and ancestor masks.
- 2: $p \leftarrow$ target probabilities from path-local GDN, convolution, and attention computation.
- 3: Retain candidate hidden states, rounded replay operands, convolution histories, and temporary KV entries.

Select one continuation

- 4: $(A, b^*) \leftarrow$ accepted materialized path and pending token from sampling with p and q .

Publish the same path across all state components

- 5: **for** each GDN layer **do**
 - 6: Replay recorded operands along A into the native recurrent state.
 - 7: Gather the convolution history for A into the native running row.
 - 8: **end for**
 - 9: Remap the accepted KV entries using the same logical-to-physical slot map.
 - 10: Select the matching hidden-state row for the next drafter input.
 - 11: Advance positions along A ; keep b^* pending until its target forward.
 - 12: **return** Emitted tokens and the updated continuation boundary.
-

paths. Their continuation policies differ. SpecLA buffers accepted factors for application in a later fused update [8]; LumoTree immediately replays recorded operands through native per-layer updates and publishes the matching convolution and attention state. Weaver also uses selected-path replay, but verifies candidates with a triangular solve [9]. Bole reconstructs accepted states from compact factors in a batched commit [30]. TreeWY can fuse prior-frontier reconstruction with the next verification while retaining its working tile [32]. Thus, LumoTree pairs path recurrence with immediate native replay and coordinated hybrid-state publication. This pays explicit accepted-path update work in exchange for returning each iteration to the native state representation. Complete-cycle measurements must include that tradeoff.

V. OPTIMIZING THE LUMOTREE VERIFICATION CYCLE

The architectural choice in Section IV is realized through the implementation changes below. Three representations must agree: logical tree nodes, temporary verification storage, and the native state used by the next forward. LumoTree carries one accepted path across these representations and reorganizes preparation and selection around it, using established techniques such as fusion and graph replay. The nontrivial constraints arise at their boundaries: changing a slot order, a rounding step, or a tied candidate can change which continuation the next iteration consumes. Table II summarizes the changes and these constraints.

A. One device-selected path drives native continuation

Selecting valid output tokens does not by itself update a hybrid model’s state. The recurrent matrix must contain the accepted updates, convolution must retain the accepted history, attention must reference the accepted KV entries, and the drafter must receive the corresponding hidden row. Reconstructing these choices separately on the host introduces

path-list preparation and synchronization between acceptance and publication.

LumoTree replaces that handoff with fixed-shape device products: accepted node indices, path lengths, emitted tokens, and the selected final row. These products feed native recurrent replay, convolution gathering, KV remapping, and drafter-row selection. Recorded operands are staged for the native per-layer updates in one captured committer graph; convolution and KV publication use the same selected path [6]. Graph capture amortizes launch orchestration, while the shared path prevents each state component from independently interpreting acceptance.

The distinction between emitted and materialized tokens must survive this handoff. A correction or bonus token may be emitted while remaining pending for the next target forward; padded path entries must not advance state. After publication, persistent state resides in the native request rows, and branch cut states remain temporary scratch. This lets prefix caching and serving graphs retain the native request lifecycle. The implementation captures the serving, drafter, and committer stages separately.

B. One slot map governs attention and publication

An ancestry mask specifies visible keys, but does not specify their physical placement or the reduction groups that process them. Interleaving masked branch columns with spine keys can change the online-softmax reduction layout even when logical visibility is unchanged. Physical ordering therefore matters both for addressing and for finite-precision execution [35].

LumoTree places the principal spine contiguously in the candidate suffix while retaining logical query-row and position order. It applies the same permutation to KV write slots and ancestry-mask key columns. Acceptance remains in logical-node space; publication translates the selected nodes through that slot map into native continuation positions [6]. Reordering

Algorithm 2 Path-parallel scan and accepted-path replay

Input: Candidate tree T and the pre-tree recurrent state for each GDN layer

Output: Recurrent outputs for all candidates; native recurrent state for the accepted path

Verify paths; keep branch state temporary

```
1: for each GDN layer reached during the target forward do
2:   Compute and record this layer's rounded node operands.
3:   for each path group, in dependency order do
4:     parallel for each ready path, head, and value tile do
5:       Load the working state tile from the pre-tree state or a saved parent cut state.
6:       for each node on this path, in ancestry order do
7:         Update the working tile with this node's operands; write its recurrent output.
8:         Save the tile if a downstream path needs this node's state.
9:       end for
10:    end for
11:  end for
12: end for
```

Finish verification and select a path

13: Complete the target forward and sampling to obtain the accepted materialized path A .

Replay only the accepted path into native recurrent state

```
14: for each GDN layer do
15:   Start from this layer's pre-tree recurrent state.
16:   for each node on  $A$ , in path order do
17:     Apply the native GDN update using this layer's recorded operands.
18:   end for
19:   Publish the resulting recurrent state to the request's native running row.
20: end for
```

TABLE II

CONCRETE IMPLEMENTATION CHANGES AROUND THE PATH VERIFIER. THE THIRD COLUMN IDENTIFIES THE COORDINATION OR NUMERICAL CONSTRAINT THAT MAKES EACH CHANGE NONTRIVIAL. WORKLOAD RATES MEASURE THEIR COMBINED EXECUTION.

Starting work or constraint	LumoTree change	What must remain consistent
Host path-list construction between acceptance and publication	Device-selected path feeds captured native replay, convolution, KV and drafter selection	All consumers select the same materialized prefix; pending and padded tokens do not advance state
Logical ancestry does not determine physical KV placement	Spine-contiguous slots with coordinated writes, mask columns and accepted-path remap	Logical query order, physical source slots and native destination positions agree
Per-node convolution histories and repeated layer preparation	Static ancestry gathers, fused ordered taps and per-layer operand buffers	Tap order, cast boundaries, history padding and layer identity
Separate score preparation and argmax/top-k operations	One shared logits tensor and fused selection into graph buffers	Both reference tie conventions, candidate order and duplicate behavior
Repeated KV staging and limited decode parallelism	Grouped query heads and split-K in the patched tree-attention kernel	Tree visibility and slot mapping across partitions; characterized reduction error

only writes, only masks, or only the final copy would leave another consumer reading a different branch. The implementation change is this coordinated mapping across verification and publication.

The patched FlashAttention-2 decode kernel combines this layout with grouped query heads that share staged KV tiles and split-K context partitions. KV reuse and partitioned attention are established techniques [23], [24]; here they are applied under the tree's visibility and slot mapping. They target repeated KV staging and insufficient parallel work at batch one. Prefill retains a native-compatible attention path. The current component tests measure repeatability and numerical error of the deployed

attention kernel; the isolated effect of spine placement remains an ablation question.

C. Precompute branch histories while preserving arithmetic

The starting tree-convolution implementation constructs candidate histories with per-node operations and repeats path-row preparation across layers. LumoTree derives static gather indices from the topology, gathers windows from the prior convolution history and candidate inputs, and fuses the tap computation. Stacked operand buffers retain each GDN layer's inputs for subsequent accepted-path replay [6].

This transformation must preserve more than ancestry. Convolution tap order and cast boundaries affect the values recorded for recurrent updates, and unused history positions must retain the intended padding. A generic reduction can compute the same real-arithmetic sum in a different floating-point order. The fused implementation therefore retains the ordered tap operations and conversion points while replacing repeated history construction with prepared data movement. Each layer still records and replays its own operands.

D. Fuse draft selection without changing proposals

At each MTP head depth, LumoTree computes one full-vocabulary logits tensor and reuses it for the principal token and sibling candidates. A fused CUDA kernel writes the spine token and three ordered candidate indices directly into reusable graph buffers, replacing separate argmax/top-k reductions and their output copies. The principal chain advances through captured MTP iterations; sibling selection reuses its scores, and suffix prediction extends selected paths within the fixed verification capacity [6].

The subtlety is that argmax and top-k do not have interchangeable tie behavior. The reference argmax chooses the lowest maximal token index, while its top-k operation orders selected ties differently. Simply taking the first top-k result as the spine token can change the proposal tree. Our fused selection preserves both reference outputs, including duplicate proposals where produced by the reference. Exact-output and captured-replay tests check this component. The optimization removes redundant score preparation and selection work while retaining the vocabulary projection.

VI. EVALUATION SETUP

Performance is evaluated on coding-agent workloads, with the task subset and serving configuration attached to every reported rate. The Qwen3.8-27B NVFP4 evidence uses SWE-bench Verified Astropy tasks on a single GB10/DGX Spark: a shared two-task comparison of three tree runs against SGLang EAGLE, plus a separate ten-task tree deployment [6], [10]. The best recorded shared-task rate supplies the headline; later runs and the larger deployment remain in the evaluation. These selected subsets are not the full benchmark. Every decode-rate result uses the pooled completed-request estimator in Eq. 5, excluding first-output latency. Recorded agent duration includes the agent loop, model service, tools, and repository operations; server startup and subsequent evaluator time are separate.

The evaluated tree has 27 draft candidates and one root, padded to 32 verification rows. A five-depth MTP head supplies 15 candidates; suffix prediction adds a six-token principal tail and rescue branches of four and two tokens. Maximum draft depth is eleven, and four post-root MTP forwards construct the head. Attention groups query heads in pairs and divides the context into four partitions. The model has 48 GDN layers and sixteen full-attention layers. All reported tree runs use this geometry and the patched FlashAttention-2 path; exact source and kernel versions are retained in the artifact.

TABLE III

ALL TEN TASKS IN THE COMPLETED CQC10 ASTROPY CASE. IDS HAVE PREFIX `ASTROPY__ASTROPY-`. TIMES ARE AGENT ELAPSED MINUTES, ROUNDED FROM SAVED RUNNER METADATA; EVALUATION IS SUBSEQUENT. THIS SELECTED DEVELOPMENT SEGMENT HAS ONE ATTEMPT PER LISTED TASK AND NO MATCHED NATIVE ARM.

Task suffix	Agent time (min)	Saved evaluation
13977	24.59	Failed tests
14096	22.99	Resolved
14182	7.23	Failed tests
14309	1.67	Resolved
14365	6.78	Failed tests
14369	27.70	Resolved
14508	41.24	Resolved
14539	18.57	Resolved
14598	25.79	Failed tests
14995	4.59	Resolved

VII. RESULTS

A. SWE-bench Verified coding-agent workload

Our workload consists of Astropy instances from the SWE-bench Verified test split. The agent uses Qwen Code 0.19.4 to inspect repositories, edit files, execute tools, and produce patches. A shared two-task comparison covers three tree runs and one SGLang EAGLE run that each produced nonempty patches on both tasks. A separate ten-task segment, Cqc10, recorded on 24 August 2026, supplies the latest complete single-configuration deployment case. The task/evaluation worker is x86-64; model serving runs on the GB10. These are full agent attempts. Table III lists every task and saved evaluator outcome in Cqc10.

a) Ten-task deployment: The served model is the RadixArk Qwen3.8-27B NVFP4 checkpoint with ModelOpt mixed quantization and bfloat16 execution dtype. The tree verifier uses the geometry and attention configuration specified in Section VI, with full-vocabulary drafting and reused draft logits. Prefix caching and full/piecewise CUDA graphs are enabled. The engine permits one active sequence, with maximum context 131,072 tokens and a 4,096-token scheduler budget. Source commit `e7af6b595a8b` and the loaded extension hash are retained in the artifact. The implementation uses the graph/cache-enabled stochastic route described above.

The proxy supplies temperature 0.6, top-p 0.95, top-k 20, minimum- p 0, and presence penalty 1.0; the engine seed is zero. The per-response output cap is 24,000 tokens, with a separate 20,000-token compaction cap and a 9,000-second agent timeout. Proxy auto-continuation is disabled. The agent runs in the task’s repository image on an internal network that permits the model proxy and blocks external egress; per-task records retain the verified network-policy fingerprint. Agent duration comes from the task runner’s elapsed-time field: it includes the agent loop and tool work, and excludes server startup and subsequent evaluation. We do not infer a completed-task speedup from model-only counters.

b) Workload observations: All ten attempts have evaluator reports: six resolved their tasks and four failed tests. Recorded

agent durations sum to 181.15 minutes, with a range of 1.67–41.24 minutes. This is an observed task ledger, not a representative SWE-bench Verified success estimate. The sum is accumulated agent time, not elapsed campaign time including startup and evaluation.

Every task used tools and produced a nonempty patch. Analysis of the recorded traces detects no degeneration in these ten runs and no malformed tool-call arguments. The degeneration heuristic jointly requires at most 200 visible characters, zero tool calls, and at least 2,000 thinking characters; none of the ten traces meets that conjunction. This screen can miss loops that continue narrating or using tools, so its zero detections establish neither universal degeneration freedom nor preserved task quality. The earlier campaign’s detected degeneration remains an adverse observation in the lineage audit.

c) *Pooled decode comparison:* For completed request r , let n_r be its output-token count, ℓ_r its server-recorded end-to-end latency, and f_r its time to first output. With R completed requests and $N = \sum_r n_r$, we use

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)} = \frac{N - R}{\sum_r \ell_r - \sum_r f_r}. \quad (5)$$

Counts and durations are pooled before division. The numerator uses the conventional $n_r - 1$ output intervals; outputs arriving in one batch can have zero arrival gaps. The denominator excludes first-output latency and between-request agent/tool work. It is accumulated request time, which counts overlapping requests separately, not isolated GPU time or elapsed campaign time.

For Cqc10, $N = 248,342$, $R = 265$, $\sum_r \ell_r = 10,667.6211$ seconds, and $\sum_r f_r = 989.8239$ seconds yield 25.63 tokens/s. The 265 requests reconcile with 256 normal requests and nine successful internal compaction requests; all recorded completion reasons are stop. The ten pre/post metric brackets are contiguous. Generation-token and completed-request counters agree with their latency-histogram populations, with no counter reset or outstanding request at the selected boundaries.

The eligible SGLang EAGLE comparator, recorded on 24 August, serves the as-shipped Qwen3.8-27B NVFP4 checkpoint with FlashInfer, using speculative steps/top- k /draft-token settings of 3/1/4. It uses the same Qwen Code version, agent harness, requested sampling settings, and network policy, with a 24,000-token response cap, a 262,144-token context limit, and an 8,192-token prefill chunk. Its two completed Astropy tasks are 12907 (resolved; 7.10 agent minutes) and 13033 (failed tests; 26.96 agent minutes). Their counters give $N = 47,854$, $R = 45$, $\sum_r \ell_r = 2,013.6344$ seconds, and $\sum_r f_r = 235.9488$ seconds, yielding 26.89 tokens/s. Both brackets contain streaming requests only. All completed engine requests are included, including internal agent traffic.

d) *Performance eligibility:* After inspecting the archived runs, we restrict the shared-task rate comparison to run-pairs in which both tasks completed and produced nonempty patches. We apply this retrospective rule to both systems and retain patches that fail tests. All three tree pairs and one of the two

TABLE IV
POOLED DECODE RATES FOR TREE VERIFICATION AND SGLANG EAGLE WITH QWEN3.8-27B NVFP4 ON GB10, USING EQ. 5. THE FIRST FOUR ROWS USE THE SAME TWO SWE-BENCH VERIFIED ASTROPY TASKS, 12907 AND 13033; EACH INCLUDED RUN PRODUCES NONEMPTY PATCHES ON BOTH TASKS; DECODER SETTINGS ARE SPECIFIED IN THE TEXT. CQC10 COVERS A SEPARATE TEN-TASK SUBSET. BOLD MARKS THE BEST RECORDED ELIGIBLE TREE RATE.

Deployment	Tasks	Requests	Tokens/s
SGLang EAGLE	2	45	26.89
Tree Sr12 (Aug 19)	2	41	29.09
Tree Cqc16 (Aug 19)	2	35	28.28
Tree Cqc15 (Aug 23)	2	53	27.27
Tree Cqc10 (Aug 24)	10	265	25.63

recovered SGLang pairs meet the rule. The other SGLang pair is excluded: task 13033 produced an empty patch after a final thinking-only response at the configured output cap, so the test harness was not invoked. Its failed outcome and original records remain in the artifact. We exclude the entire pair from this performance comparison, rather than retaining only its successful task. The rates therefore characterize patch-producing runs, not performance over all attempted tasks.

e) *Best recorded shared-task result:* We select task IDs 12907 and 13033 because both have completed SGLang records, then report all three recovered split-K tree runs on that pair. Sr12 and Cqc16 ran on 19 August; Cqc15 ran on 23 August. All three use the same tree geometry, full-vocabulary drafting, and grouped split-K attention configuration as Cqc10. Runtime checks confirm the specialized attention kernel in all sixteen full-attention layers, with no fallback. They share the loaded attention extension but differ in Python patch revisions. Each uses a 32,768-token response cap, compared with the eligible SGLang run’s 24,000. All 41 Sr12 responses and all 45 eligible SGLang responses are at most 20,000 tokens, below both configured caps; no response uses the differing output allowance. All Sr12 requests terminate with stop.

The best shared-task tree observation is Sr12: $N = 25,027$, $R = 41$, $\sum_r \ell_r = 989.2612$ seconds, and $\sum_r f_r = 130.4422$ seconds give **29.09 tokens/s**, 8.18% above the eligible SGLang rate under Eq. 5. Later observations are 28.28 for Cqc16 and 27.27 for Cqc15; Table IV retains them alongside the best result. All selected task brackets pass the same population and idle-boundary checks. Every row on the shared pair has one resolved task and one failed task. All six selected tree attempts used tools and produced nonempty patches, with no detected degeneration or malformed tool-call arguments under the recorded checks. Every selected tree request finished with stop, without a recorded response-cap hit.

The tree headline is its best recorded rate on the shared pair, not the latest run or Sr12’s entire workload. Sr12 completed four tasks in total, two resolved and two failed; pooling all four gives 26.21 tokens/s. Cqc16’s subsequent task 13236 exhibited degeneration, and Cqc15’s subsequent task 13398 lacks a completed evaluation. Those adverse records remain in the artifact and outside this explicitly two-task comparison.

The shared-task comparison evaluates the complete decoding systems, including their proposal topology, draft budget, and serving optimizations. These method-specific choices are part of the comparison. The records use the same requested sampling settings and agent harness. The performance table retains every eligible run-pair: three tree observations and one SGLang observation. This retrospective subset does not establish performance over all attempts or each system’s optimum over draft lengths and candidate budgets. Repeated runs with a predeclared tuning protocol would measure that performance envelope; a component ablation would separately isolate the contribution of tree verification. The artifact retains the original metric brackets, source-bound configuration and behavior records, and reproducible pooled reductions.

f) Selection and comparison limits: Cqc10 contains the ten remaining tasks of an interrupted sixteen-task development campaign. Earlier segments changed source revisions and output caps and included degeneration, incomplete output, and budget-capped attempts. Source-bound audit records of those failures, retries, and missing evaluations remain in the artifact; the original records remain at their recorded source paths. We do not relabel the completed segment as the entire campaign, combine those segments into a single-configuration benchmark score, or drop unsuccessful attempts from a claimed benchmark denominator. The tasks were selected from one repository, and this deployment has one recorded boot without a matched ten-task native run. Separate targeted native probes change other serving settings and do not supply that comparator. Consequently, these records demonstrate an implemented verifier serving real coding-agent work, but establish no native-versus-tree task-speed or quality advantage. A current, matched task campaign is the remaining experiment for that claim.

g) Existing comparator experiments: Earlier SGLang EAGLE trials and native MTP controls remain in the repository. The earlier native campaign predates the NVFP4 deployment; later NVFP4 native probes cover targeted tasks with different serving settings. DSpark measurements use synthetic requests, and a later DFlash2 check establishes boot and response handling rather than agent-task performance. These records are distinguished from the pooled SWE comparison above; their superseded or non-agent rates are not additional performance results for the current verifier.

B. Numerical and candidate-selection validation

The fused full-vocabulary selection test compares the two output tensors against the reference argmax and top-k operations. Across 1,368 input cases and five block-count settings (6,840 configurations), it records zero byte mismatches; planted negative controls fire. A separate captured four-level selection graph completes 24 replays with zero mismatches. These results validate candidate-selection outputs for the tested inputs and graph behavior, not end-to-end task equivalence [6].

Attention tests use the same kernel build as the ten-task deployment. They show bit-identical repeated output and log-sum-exp values within each of sixteen determinism cases, with matching digests across two processes. On synthetic

query/key/value tensors drawn at scales measured from banked model operands, 93.31% of output elements agree with the reference kernel within two ULP, the maximum absolute output difference is 0.00390625, and maximum log-sum-exp disagreement is four ULP. All nine declared checks pass, including comparisons against a float64 dense reference and checks for non-finite disagreements. These are attention-component bounds; split-K changes rounding and is not claimed to be byte-identical to the reference kernel or to preserve every downstream token.

Runtime checks confirm that the evaluated runs use the specialized attention kernel in all sixteen attention layers, fused selection, device acceptance, and captured state publication. The task outcomes and trace analysis in Section VII-A evaluate the composed system. These component and workload observations address different levels of behavior; neither establishes a full-model conditional-distribution proof.

VIII. DISCUSSION AND REMAINING EVIDENCE

The design couples tree geometry, physical layout, and continuation publication. Optimizing any one of these in isolation can move work elsewhere or invalidate another stage’s mapping. The verifier uses transient cut states, static graph buffers, and native persistent rows for different purposes; conflating them would obscure both memory cost and state ownership.

Sequential replay and compact reconstruction remain alternative implementation choices. Appendix B records initial runs of the TreeWY and Weaver author components on GB10. These checks establish executable comparison candidates; they do not compare complete systems against LumoTree. A direct performance comparison requires compatible models, matched workloads, and each system’s actual execution path.

For long-running agents, task completion and task wall time are the relevant application outcomes. More decoded tokens can reflect additional reasoning or a longer tool loop. The pooled decode comparison is therefore accompanied by task identities, outcomes, agent/tool policy, and terminal budgets. It describes the evaluated deployments and the stated retrospective patch-producing subset; it is not a task-completion speedup estimate.

Component validation and whole-task performance answer different questions. The selection test checks exact candidate outputs; the split-K test checks deterministic bounded rounding; the workload tests exercise the composed system. A broader quality claim requires held-out tasks and repetitions. Isolating an optimization’s application benefit requires toggling that component within the same eligible serving configuration and task protocol.

IX. CONCLUSION

LumoTree executes recurrent paths in parallel, reuses each working state tile across path updates, and publishes one accepted continuation across the hybrid model. A shared descriptor coordinates attention visibility, branch-local GDN computation, convolution history, device probability arithmetic,

and native continuation. Sequential scan and accepted-path replay make the state lifetime and arithmetic choices explicit; cached state, recomputation, physical layout, and independent-work batching expose the main optimization tradeoffs.

On two shared SWE-bench Verified Astropy tasks with Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both tasks, the best recorded tree run attains 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. Later tree runs on the same pair attain 28.28 and 27.27; the separate ten-task deployment attains 25.63. All ten attempts in that larger deployment terminated with nonempty patches and no degeneration detected by the recorded trace checks. The common rate definition compares the complete decoding systems; draft budgets, selection scope, and task outcomes are reported alongside the rates.

The contribution is a concrete verifier design and its implementation mechanisms, supported by component validation and workload-specific observations. A broader performance claim requires additional tasks and repeated runs with a declared tuning protocol. The present results do not establish broader task-quality preservation, full-model sequential equivalence, or general superiority over SGLang or the published compact-state systems.

APPENDIX A REPRODUCIBILITY AND CLAIM BOUNDARIES

The artifact records source revisions, effective serving settings, kernel identities, tree geometry, graph and cache configuration, and original task metadata. Dated development records are retained separately from the evidence supporting the evaluated method.

All workload rates use the completed-request pooled decode estimator in Eq. 5. Token totals and latency sums come from the same completed-request population. First-output latency, tool execution, and evaluator time are separate quantities. Earlier global-token/subset-wall proxies and local-document timing do not supply paper performance claims. The selection ledger preserves every included and excluded run-pair and the unchanged unfiltered measurements.

Algebraic correctness concerns the real-arithmetic recurrence. *Numerical agreement* concerns particular kernels and references. *Sampler correctness* concerns the acceptance/residual law for the supplied probabilities. *Serving correctness* additionally requires coherent state across continuation and cache operations [2], [6], [26]. Equal candidate outputs, bounded attention error, and observed task completion address different parts of that chain.

For target probabilities p and proposal probabilities q , ordinary rejection sampling combines accepted mass $\min(p, q)$ with residual mass proportional to $(p - q)_+$. This gives the intended target marginal only for the probabilities actually supplied [1], [2]. A correct abstract sampling rule cannot compensate for stale state or different target logits. The paper therefore does not promote component checks or task outcomes into a general distribution-preservation claim.

APPENDIX B AUTHOR-CODE COMPONENT CHECKS

To prepare direct mechanism comparisons, we ran pinned TreeWY and Weaver author components on the same GB10 host, using PyTorch 2.10.0a0 and Triton 3.6.0 with CUDA 13.1. Import adapters load the pinned modules without changing kernel arithmetic, test inputs, or assertions. These runs exercise isolated components rather than the authors’ complete serving stacks.

For TreeWY, revision `b073ed6c`, all 39 selected author tests pass: sixteen GPU cases cover chain/tree forward computation, accepted-state reconstruction, sentinel behavior, and tree graph replay; 23 reference cases check the algebraic implementation. Full vLLM layer and model integration tests are outside this subset [32].

For Weaver, revision `aeac03f0`, the author’s precision-comparison script completes nine synthetic cases (three tree shapes and three seeds) and two verifier-timing configurations [39]. It compares precision modes, without a pass/fail tolerance or complete commit cost. The pinned source includes both block-quadratic and fused verifier routes; its serving backend records operands and replays the accepted path. The artifact retains the exact modules, adapters, environment, and raw outputs. These initial runs supply neither a cross-method speed ratio nor an end-to-end qualification.

APPENDIX C REMAINING WORKLOAD COMPARISONS

A matched native-MTP/tree comparison would freeze the checkpoint, task cohort, agent harness, sampling, tool policy, and budgets, then report outcomes and completion time over every attempted task. Each method may use its own speculative depth and serving optimizations; the tuning protocol and selected configuration must be explicit. Confirmation should predeclare performance eligibility and retain unsuccessful attempts in the outcome report.

The next component comparison pairs the current path verifier/native replay with the pinned TreeWY and Weaver implementations on identical operands and trees. It must compare outputs and the accepted continuation before timing, include metadata and commitment, and account for TreeWY’s prior-step commit fused into the next verification. Allocated scratch, state traffic, and register spills are separate measurements. SpecLA and Bole inform the mechanism comparison; running their author systems remains contingent on obtaining verified implementations. Layout attribution needs a controlled intervention with unchanged logical candidates and correctly paired KV/mask remapping. These component diagnoses remain separate from workload speed.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>

- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification," 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving mamba2 with delta rule," 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "Stree: Speculative tree decoding for hybrid state-space models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, "Lumo FlyWheel implementation and experiment archive," https://github.com/MaCoredroid/Lumo_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c, 2026, revision evidence snapshot; individual campaigns retain their own run identities.
- [7] —, "Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native," https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html, 2026, author experiment archive; accessed 2026-09-21.
- [8] Z. Wang, X. Han, Z. Yang, F. Liu, X. Li, R. Gu, S. Zhong, and C. Tian, "SpecLA: Efficient Speculative Decoding for Linear-Attention Models," 2026. [Online]. Available: <https://arxiv.org/abs/2607.16673v1>
- [9] Y. Oda, R. Mathieu, R. Knyazhitskiy, and A. Chakhvadze, "Trees from Marginals: Autoregressive drafting with factorized priors," 2026, version 2, July 12, 2026; first submitted July 7, 2026. [Online]. Available: <https://arxiv.org/abs/2607.06763v2>
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [11] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable, robust, and hardware-aware speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>
- [12] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle-2: Faster inference of language models with dynamic draft trees," 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [13] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "Opt-tree: Speculative decoding with adaptive draft tree structure," 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [14] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test," 2025. [Online]. Available: <https://arxiv.org/abs/2503.01840>
- [15] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block Diffusion for Flash Speculative Decoding," 2026. [Online]. Available: <https://arxiv.org/abs/2602.06036>
- [16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [17] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [18] R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali, "Marconi: Prefix Caching for the Era of Hybrid LLMs," 2025. [Online]. Available: <https://arxiv.org/abs/2411.19379>
- [19] SGLang Team, "Hybrid Models Meet SGLang: More than Full Attention," <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [20] SGLang Project, "Qwen3-Next SGLang documentation," <https://lmsys.org/mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [21] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-Decoding for long-context inference," <https://pytorch.org/blog/flash-decoding/>, 2023, october 13, 2023.
- [22] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasicki, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," 2025. [Online]. Available: <https://arxiv.org/abs/2501.01005>
- [23] J. Yao, K. Chen, K. Zhang, J. You, B. Yuan, Z. Wang, and T. Lin, "DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference," 2025. [Online]. Available: <https://arxiv.org/abs/2404.00242>
- [24] Z. Pan, Y. Ding, Y. Guan, Z. Wang, Z. Yu, X. Tang, Y. Wang, and Y. Ding, "FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference," in *Proceedings of Machine Learning and Systems*, vol. 7, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/hash/96894468eb44631a32d7ebd56f9892c7-Abstract-Conference.html
- [25] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [26] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [27] Y. Wu, Y. Dukler, M. Trager, A. Achille, W. Xia, and S. Soatto, "Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding," in *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, vol. 262. PMLR, 2024, pp. 292–304. [Online]. Available: <https://proceedings.mlr.press/v262/wu24a.html>
- [28] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, "Specmamba: Accelerating mamba inference on fpga with speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [29] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, "Component-aware self-speculative decoding in hybrid language models," 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [30] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, "Bole: Efficient Tree Speculation for Hybrid-Attention Language Models," 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [31] S. M. Gantasala, "TreeWY: Speculative Verification for Gated DeltaNet Hybrids," 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [32] —, "TreeWY: Tree Speculative Decoding for Hybrid GDN Models," <https://github.com/vllm-project/vllm/issues/54080>, 2026, author implementation RFC; linked source revision b073ed6cacfa1cf5ae23111854729e662a72a6d1; accessed September 26, 2026.
- [33] Dao AI Lab and NVIDIA, "ReplaySSM: Cache SSM Inputs Instead of State," <https://dao-lab.ai/blog/2026/replayssm/>, 2026, author technical report; accessed 2026-09-26.
- [34] X. Yang, C. Peng, Y. Zhao, L. Zeng, A. Hu, J. Yang, S. Wang, J. Lv, X. Liang, C. Yang, J. Liu, and Y. Qiu, "OneLA: Scaling Linear-Attention Decoding to Large Beams in Generative Recommendation," 2026. [Online]. Available: <https://arxiv.org/abs/2609.12399v1>
- [35] H. He, "Defeating Nondeterminism in LLM Inference," <https://thinkngmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>, 2025, thinking Machines Lab, September 10, 2025.
- [36] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications," 2025. [Online]. Available: <https://arxiv.org/abs/2411.04975v3>
- [37] Snowflake Arctic Inference, "Suffix Decoding," <https://arcticinference.readthedocs.io/en/latest/suffix-decoding.html>, 2026, implementation documentation; accessed September 24, 2026.
- [38] Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen, and J. Zhang, "SAM Decoding: Speculative Decoding via Suffix Automaton," 2024. [Online]. Available: <https://arxiv.org/abs/2411.10666v3>
- [39] Mirai, "Weaver SGLang implementation," <https://github.com/trymirai/sglang/tree/aeac03f0d4c8789559411be95c5c127bdf24d1c>, 2026, pinned author source; accessed September 26, 2026.